

Pre Test Advanced Stored Function

Nama : Raihan Muhammad Riswandi

NIM: 24110500001

Soal 1

Tiga jenis parameter dalam stored functions PostgreSQL adalah:

1. IN (default): Parameter input yang nilainya diberikan saat function dipanggil
2. OUT: Parameter output yang berfungsi sebagai return value tambahan
3. INOUT: Parameter yang berfungsi sebagai input sekaligus output

Soal 2

Syntax dasar:

```
```sql
```

```
CREATE FUNCTION get_students()
```

```
RETURNS TABLE(student_id INTEGER, student_name TEXT, gpa NUMERIC)
```

```
LANGUAGE plpgsql AS $$
```

```
BEGIN
```

```
 RETURN QUERY SELECT s.id, s.name, s.gpa FROM students s;
```

```
END; $$;
```

```
```
```

Soal 3

RETURN QUERY digunakan untuk mengembalikan hasil query secara keseluruhan, sedangkan RETURN NEXT digunakan untuk mengembalikan satu baris hasil perhitungan atau manipulasi data. RETURN NEXT lebih fleksibel untuk data yang diproses secara individual.

Soal 4

Function dengan OUT parameters tidak memerlukan RETURNS clause karena PostgreSQL secara otomatis menganggap return type sebagai composite type yang terdiri dari semua OUT parameters. Mekanisme di balik layar: PostgreSQL membuat tuple implicit dari OUT parameters sebagai return value.

Soal 5

CASE statement lebih baik digunakan ketika terdapat banyak kondisi yang saling eksklusif berdasarkan nilai yang sama. Contoh: klasifikasi nilai mahasiswa (A, B, C) berdasarkan range angka. CASE lebih readable dan efisien dibanding nested IF untuk skenario seperti ini.

Soal 6

Table functions dapat digunakan dalam FROM clause dengan memanggil function seperti tabel: `SELECT \* FROM my\_table\_function()`. Keuntungan untuk reporting: dapat

mengemas logika kompleks dalam function dan menggunakannya sebagai sumber data yang reusable, menyederhanakan query laporan.

Soal 7

Masalah: Jika parameter counter bernilai NULL, operasi `counter + 1` akan menghasilkan NULL (karena operasi aritmetika dengan NULL). Solusi: Tambahkan penanganan NULL dengan COALESCE:

```
```sql
```

```
CREATE FUNCTION update_counter(counter INOUT INTEGER)
```

```
LANGUAGE plpgsql AS $$
```

```
BEGIN
```

```
 counter := COALESCE(counter, 0) + 1;
```

```
END; $$;
```

```
```
```